

Reflection Report — ShopEasy Term Project

Software Testing Course

1. Which bugs did each technique reveal?

Most techniques confirmed that the code was correct rather than finding real bugs. But mutation testing was a different experience — it was my first time using PIT and I was surprised by what it found. It changed a boundary condition in ShoppingCart from `quantity <= 0` to `quantity < 0`, and my tests did not notice. The same thing happened in Product with zero price and zero stock. These are not bugs in the production code, but they showed me that my tests were not as thorough as I thought. It was a bit uncomfortable to see, but also useful.

2. Which technique was most effective per unit of effort?

Specification-based testing was the easiest one. The method contracts were already written in the Javadoc, so I just needed to turn them into test cases. Property-based testing took more time to understand at first, but once I got it, it was very useful — jqwik tested each property 1000 times with random inputs, which is much better than writing a few fixed cases by hand. Mocking felt strange at first. I kept thinking "how are we testing real behavior if we're replacing everything with fakes?" But after writing the OrderProcessor tests, I understood the point — you want to test one thing at a time, not the whole system at once.

3. How would you classify your test suite?

Almost all of my tests are unit tests, which makes sense for this project since there are no databases or external services. According to the testing pyramid, this is a reasonable shape — many small fast unit tests at the bottom. What is missing is better boundary testing for classes like Product and CartItem. Also, OrderProcessor is only tested with mocks, so there is no test that checks the full flow with real implementations. The pyramid is missing its middle layer.

4. If you had one more week, what would you test next?

I would first add tests for the boundary values that PIT found — for example, creating a Product with price zero or stock zero to make sure these edge cases are properly handled. Then I would try writing some tests for OrderProcessor without mocks, using simple real implementations, to see if the components work correctly together. These integration-style tests would catch bugs that mocking hides.